

From Finite Fields to Reed-Solomon Codes

Yunghsiang S. Han (韓永祥)

yunghsiangh@gmail.com

(深圳) 高等研究院



电子科技大学

University of Electronic Science and Technology of China

December 27, 2021

Why Reed-Solomon Codes

Have been used in

- Space communication: Voyager space probe, Mars Pathfinder, Galileo, Mars Exploration Rover, and Cassini missions

Why Reed-Solomon Codes

Have been used in

- Space communication: Voyager space probe, Mars Pathfinder, Galileo, Mars Exploration Rover, and Cassini missions
- Data transmission technologies: DSL, WiMAX, cellular phone system, and video conference over Internet

Why Reed-Solomon Codes

Have been used in

- Space communication: Voyager space probe, Mars Pathfinder, Galileo, Mars Exploration Rover, and Cassini missions
- Data transmission technologies: DSL, WiMAX, cellular phone system, and video conference over Internet
- Broadcast systems: DVB and ATSC

Why Reed-Solomon Codes

Have been used in

- Space communication: Voyager space probe, Mars Pathfinder, Galileo, Mars Exploration Rover, and Cassini missions
- Data transmission technologies: DSL, WiMAX, cellular phone system, and video conference over Internet
- Broadcast systems: DVB and ATSC
- Storage systems: RAID 6 and distributed storage (data centers)

Why Reed-Solomon Codes

Have been used in

- Space communication: Voyager space probe, Mars Pathfinder, Galileo, Mars Exploration Rover, and Cassini missions
- Data transmission technologies: DSL, WiMAX, cellular phone system, and video conference over Internet
- Broadcast systems: DVB and ATSC
- Storage systems: RAID 6 and distributed storage (data centers)
- Consumer technologies: CDs, DVDs, Blu-ray Discs, and Bar codes (such as PDF417, QR Codes)

The Field of Rational Numbers

- Let $Q = \{\frac{p}{q} | q \neq 0, p, q \text{ are integers}\}$
- Define two operations: Addition: $a + b$, Multiplication: $(a \cdot b)$

The Field of Rational Numbers

- Let $Q = \{\frac{p}{q} | q \neq 0, p, q \text{ are integers}\}$
- Define two operations: Addition: $a + b$, Multiplication: $(a \cdot b)$
- $3/2 + 4/3 = 17/6$, $3/2 \cdot 4/3 = 2$

The Field of Rational Numbers

- Let $Q = \{\frac{p}{q} | q \neq 0, p, q \text{ are integers}\}$
- Define two operations: Addition: $a + b$, Multiplication: $(a \cdot b)$
- $3/2 + 4/3 = 17/6$, $3/2 \cdot 4/3 = 2$
- $0 + a = a \longrightarrow 0$ is **additive identity**
- $1 \cdot a = a \longrightarrow 1$ is **multiplicative identity**

The Field of Rational Numbers

- Let $Q = \{\frac{p}{q} | q \neq 0, p, q \text{ are integers}\}$
- Define two operations: Addition: $a + b$, Multiplication: $(a \cdot b)$
- $3/2 + 4/3 = 17/6$, $3/2 \cdot 4/3 = 2$
- $0 + a = a \longrightarrow 0$ is **additive identity**
- $1 \cdot a = a \longrightarrow 1$ is **multiplicative identity**
- $a + (-a) = 0 \longrightarrow a$ and $-a$ **additive inverse** (subtraction)
- If $p \neq 0$, $\frac{p}{q} \cdot \frac{q}{p} = 1 \longrightarrow \frac{p}{q}$ and $\frac{q}{p}$ **multiplicative inverse** (division)

The Field of Rational Numbers

- Let $Q = \{\frac{p}{q} | q \neq 0, p, q \text{ are integers}\}$
- Define two operations: Addition: $a + b$, Multiplication: $(a \cdot b)$
- $3/2 + 4/3 = 17/6$, $3/2 \cdot 4/3 = 2$
- $0 + a = a \longrightarrow 0$ is **additive identity**
- $1 \cdot a = a \longrightarrow 1$ is **multiplicative identity**
- $a + (-a) = 0 \longrightarrow a$ and $-a$ **additive inverse** (subtraction)
- If $p \neq 0$, $\frac{p}{q} \cdot \frac{q}{p} = 1 \longrightarrow \frac{p}{q}$ and $\frac{q}{p}$ **multiplicative inverse** (division)
- $4/3 = 4 \cdot \frac{1}{3} = 4 \cdot 3^{-1} = \frac{4}{3}$

► Click on to the definition of fields

An Example of Prime Fields

- Let $F = \{0, 1, 2, 3, 4\}$
- Define two operations: Addition: $a + b \bmod 5$, Multiplication: $(a \cdot b) \bmod 5$

An Example of Prime Fields

- Let $F = \{0, 1, 2, 3, 4\}$
- Define two operations: Addition: $a + b \bmod 5$, Multiplication: $(a \cdot b) \bmod 5$
- $3 + 4 \bmod 5 = 7 \bmod 5 = 2$
 $3 \cdot 4 \bmod 5 = 12 \bmod 5 = 2$

An Example of Prime Fields

- Let $F = \{0, 1, 2, 3, 4\}$
- Define two operations: Addition: $a + b \bmod 5$, Multiplication: $(a \cdot b) \bmod 5$
- $3 + 4 \bmod 5 = 7 \bmod 5 = 2$
 $3 \cdot 4 \bmod 5 = 12 \bmod 5 = 2$
- $0 + a \bmod 5 = a \longrightarrow 0$ is **additive identity**
- $1 \cdot a \bmod 5 = a \longrightarrow 1$ is **multiplicative identity**

An Example of Prime Fields

- Let $F = \{0, 1, 2, 3, 4\}$
- Define two operations: Addition: $a + b \bmod 5$, Multiplication: $(a \cdot b) \bmod 5$
- $3 + 4 \bmod 5 = 7 \bmod 5 = 2$
 $3 \cdot 4 \bmod 5 = 12 \bmod 5 = 2$
- $0 + a \bmod 5 = a \longrightarrow 0$ is **additive identity**
- $1 \cdot a \bmod 5 = a \longrightarrow 1$ is **multiplicative identity**
- $a + (5 - a) \bmod 5 = 0 \longrightarrow a$ and $5 - a$ ($-a$) **additive inverse**
- $3 + (-2) = 3 + (5 - 2) = 6 = 1 \bmod 5$

An Example of Prime Fields

- Let $F = \{0, 1, 2, 3, 4\}$
- Define two operations: Addition: $a + b \bmod 5$, Multiplication: $(a \cdot b) \bmod 5$
- $3 + 4 \bmod 5 = 7 \bmod 5 = 2$
 $3 \cdot 4 \bmod 5 = 12 \bmod 5 = 2$
- $0 + a \bmod 5 = a \longrightarrow 0$ is **additive identity**
- $1 \cdot a \bmod 5 = a \longrightarrow 1$ is **multiplicative identity**
- $a + (5 - a) \bmod 5 = 0 \longrightarrow a$ and $5 - a$ ($-a$) **additive inverse**
- $3 + (-2) = 3 + (5 - 2) = 6 = 1 \bmod 5$
- $a \cdot a^{-1} \bmod 5 = 1 \longrightarrow a$ and a^{-1} **multiplicative inverse**
- $3 \cdot 3^{-1} = 3 \cdot 2 = 1 \bmod 5$

Multiplicative Inverse of Prime Fields

- Let $F = \{0, 1, \dots, p-1\}$, where p is a prime. Then F is a prime field

Multiplicative Inverse of Prime Fields

- Let $F = \{0, 1, \dots, p-1\}$, where p is a prime. Then F is a prime field
- Let $p = 113$, What is 101^{-1} ? By **Extended Euclidean algorithm**

Multiplicative Inverse of Prime Fields

- Let $F = \{0, 1, \dots, p-1\}$, where p is a prime. Then F is a prime field
- Let $p = 113$, What is 101^{-1} ? By **Extended Euclidean algorithm**
- Euclidean algorithm is a recursive technology to find the **greatest common divisor (GCD)** of two numbers or two polynomials

Multiplicative Inverse of Prime Fields

- Let $F = \{0, 1, \dots, p-1\}$, where p is a prime. Then F is a prime field
- Let $p = 113$, What is 101^{-1} ? By **Extended Euclidean algorithm**
- Euclidean algorithm is a recursive technology to find the **greatest common divisor (GCD)** of two numbers or two polynomials
- The Euclidean algorithm is as follows. Let a and b , $a > b$, represent the two integers Divide a by b . If the remainder, r , is zero, then $\text{GCD } d = b$. If the remainder is not zero, then replace a with b , replace b with r , and repeat

Extended Euclidean Algorithm

- Extended Euclidean algorithm records $f_i a + g_i b = r_i$ for iteration i .

$$g_i b = r_i \bmod a$$

Extended Euclidean Algorithm

- Extended Euclidean algorithm records $f_i a + g_i b = r_i$ for iteration i .

$$g_i b = r_i \bmod a$$

- Define q_i to be the quotient that is produced by dividing r_{i-2} by r_{i-1} . Then, for $i \geq 1$,

$$\begin{aligned} r_i &= r_{i-2} - q_i r_{i-1}, & f_{-1} &= g_0 = 1 \\ f_i &= f_{i-2} - q_i f_{i-1}, & f_0 &= g_{-1} = 0 \\ g_i &= g_{i-2} - q_i g_{i-1}, & r_{-1} &= a, \quad r_0 = b \end{aligned} \tag{1}$$

Extended Euclidean Algorithm

- Extended Euclidean algorithm records $f_i a + g_i b = r_i$ for iteration i .

$$g_i b = r_i \pmod{a}$$

- Define q_i to be the quotient that is produced by dividing r_{i-2} by r_{i-1} . Then, for $i \geq 1$,

$$\begin{aligned} r_i &= r_{i-2} - q_i r_{i-1}, & f_{-1} &= g_0 = 1 \\ f_i &= f_{i-2} - q_i f_{i-1}, & f_0 &= g_{-1} = 0 \\ g_i &= g_{i-2} - q_i g_{i-1}, & r_{-1} &= a, \quad r_0 = b \end{aligned} \tag{1}$$



$$\begin{aligned} r_1 = 12 &= 113 - 1 \cdot 101, & f_1 &= 1 - 1 \cdot 0 = 1, & g_1 &= 0 - 1 \cdot 1 = -1 \\ r_2 = 5 &= 101 - 8 \cdot 12, & f_2 &= 0 - 8 \cdot 1 = -8, & g_2 &= 1 - 8 \cdot (-1) = 9 \\ r_3 = 2 &= 12 - 2 \cdot 5, & f_3 &= 1 - 2 \cdot (-8) = 17, & g_3 &= -1 - 2 \cdot 9 = -19 \\ r_4 = \boxed{1} &= 5 - 2 \cdot 2, & f_4 &= -8 - 2 \cdot 17 = -42, & g_4 &= 9 - 2 \cdot (-19) = \boxed{47} \end{aligned}$$

Examples of Finite Fields

$GF(2)$						
+	0	1		·	0	1
0	0	1		0	0	0
1	1	0		1	0	1

$GF(3)$								
+	0	1	2		·	0	1	2
0	0	1	2		0	0	0	0
1	1	2	0		1	0	1	2
2	2	0	1		2	0	2	1

Binary Field Arithmetic

- We consider **polynomials over field $GF(2)$**

Binary Field Arithmetic

- We consider **polynomials over field $GF(2)$**
- Let $f(x) = \sum_{i=0}^n f_i x^i$ and $g(x) = \sum_{i=0}^m g_i x^i$, where $f_i, g_i \in GF(2)$

Binary Field Arithmetic

- We consider **polynomials over field $GF(2)$**
- Let $f(x) = \sum_{i=0}^n f_i x^i$ and $g(x) = \sum_{i=0}^m g_i x^i$, where $f_i, g_i \in GF(2)$
- $f(x) + g(x) \equiv f(x) + g(x)$ with coefficients modulo by 2
- $f(x) \cdot g(x) \equiv f(x) \cdot g(x)$ with coefficients modulo by 2
- $f(x) \cdot 0 = 0$

Binary Field Arithmetic

- We consider **polynomials over field $GF(2)$**
- Let $f(x) = \sum_{i=0}^n f_i x^i$ and $g(x) = \sum_{i=0}^m g_i x^i$, where $f_i, g_i \in GF(2)$
- $f(x) + g(x) \equiv f(x) + g(x)$ with coefficients modulo by 2
- $f(x) \cdot g(x) \equiv f(x) \cdot g(x)$ with coefficients modulo by 2
- $f(x) \cdot 0 = 0$
- $f(x)$ is said to be **irreducible** if it is not divisible by any polynomial over $GF(2)$ of degree less than n but greater than zero

Binary Field Arithmetic

- We consider **polynomials over field $GF(2)$**
- Let $f(x) = \sum_{i=0}^n f_i x^i$ and $g(x) = \sum_{i=0}^m g_i x^i$, where $f_i, g_i \in GF(2)$
- $f(x) + g(x) \equiv f(x) + g(x)$ with coefficients modulo by 2
- $f(x) \cdot g(x) \equiv f(x) \cdot g(x)$ with coefficients modulo by 2
- $f(x) \cdot 0 = 0$
- $f(x)$ is said to be **irreducible** if it is not divisible by any polynomial over $GF(2)$ of degree less than n but greater than zero
- $x^2, x^2 + 1, x^2 + x$ are reducible over $GF(2)$, $x + 1, x^2 + x + 1, x^3 + x + 1$ are irreducible over $GF(2)$

Binary Field Arithmetic

- We consider **polynomials over field $GF(2)$**
- Let $f(x) = \sum_{i=0}^n f_i x^i$ and $g(x) = \sum_{i=0}^m g_i x^i$, where $f_i, g_i \in GF(2)$
- $f(x) + g(x) \equiv f(x) + g(x)$ with coefficients modulo by 2
- $f(x) \cdot g(x) \equiv f(x) \cdot g(x)$ with coefficients modulo by 2
- $f(x) \cdot 0 = 0$
- $f(x)$ is said to be **irreducible** if it is not divisible by any polynomial over $GF(2)$ of degree less than n but greater than zero
- $x^2, x^2 + 1, x^2 + x$ are reducible over $GF(2)$, $x + 1, x^2 + x + 1, x^3 + x + 1$ are irreducible over $GF(2)$
- $(x + 1)(x + 1) = 1 + (1 + 1)x + x^2 = 1 + x^2$

Binary Field Arithmetic

- We consider **polynomials over field $GF(2)$**
- Let $f(x) = \sum_{i=0}^n f_i x^i$ and $g(x) = \sum_{i=0}^m g_i x^i$, where $f_i, g_i \in GF(2)$
- $f(x) + g(x) \equiv f(x) + g(x)$ with coefficients modulo by 2
- $f(x) \cdot g(x) \equiv f(x) \cdot g(x)$ with coefficients modulo by 2
- $f(x) \cdot 0 = 0$
- $f(x)$ is said to be **irreducible** if it is not divisible by any polynomial over $GF(2)$ of degree less than n but greater than zero
- $x^2, x^2 + 1, x^2 + x$ are reducible over $GF(2)$, $x + 1, x^2 + x + 1, x^3 + x + 1$ are irreducible over $GF(2)$
- $(x + 1)(x + 1) = 1 + (1 + 1)x + x^2 = 1 + x^2$
- $x^3 + x + 1 | x^7 + 1$, i.e., $x^7 + 1 = (x^4 + x^2 + x + 1)(x^3 + x + 1)$

Extension Finite Fields

- Let $f(x)$ be an irreducible polynomial over $GF(2)$
- We can construct a finite field based on $f(x)$ that is similar to the prime p in a prime field

Extension Finite Fields

- Let $f(x)$ be an irreducible polynomial over $GF(2)$
- We can construct a finite field based on $f(x)$ that is similar to the prime p in a prime field
- Let $F = \{g(x) \mid \deg(g(x)) < m\}$, number of elements in F is 2^m

Extension Finite Fields

- Let $f(x)$ be an irreducible polynomial over $GF(2)$
- We can construct a finite field based on $f(x)$ that is similar to the prime p in a prime field
- Let $F = \{g(x) \mid \deg(g(x)) < m\}$, number of elements in F is 2^m
- Addition: $a(x) + b(x) \bmod f(x)$, Multiplication: $a(x) \cdot b(x) \bmod f(x)$

Extension Finite Fields

- Let $f(x)$ be an irreducible polynomial over $GF(2)$
- We can construct a finite field based on $f(x)$ that is similar to the prime p in a prime field
- Let $F = \{g(x) \mid \deg(g(x)) < m\}$, number of elements in F is 2^m
- Addition: $a(x) + b(x) \bmod f(x)$, Multiplication: $a(x) \cdot b(x) \bmod f(x)$
- F along with two operations is a finite field $GF(2^m)$

Examples of Finite Fields

$$f(x) = 1 + x + x^2$$

$$f(\alpha) = 1 + \alpha + \alpha^2 = 0$$

Representations			
0	0	00	0
1	1	10	2
α	α	01	1
α^2	$1 + \alpha$	11	3

$GF(4) = GF(2^2)$										
+	0	1	2	3		·	0	1	2	3
0	0	1	2	3		0	0	0	0	0
1	1	0	3	2		1	0	3	1	2
2	2	3	0	1		2	0	1	2	3
3	3	2	1	0		3	0	2	3	1

Property of Finite Fields

- All nonzero elements in $GF(2^m)$ are roots of $x^{2^m-1} + 1$

[◀ Click for proof](#)

Property of Finite Fields

- All nonzero elements in $GF(2^m)$ are roots of $x^{2^m-1} + 1$ [Click for proof](#)
- $x^{2^m-1} + 1 = (x+1)(1+x+x^2+\cdots+x^{2^m-2})$ such that all nonzero elements except 1 are roots of

$$(1+x+x^2+\cdots+x^{2^m-2}) = \sum_{i=0}^{2^m-2} x^i$$

Primitive Polynomials

- Let $\alpha \in GF(2^m)$ such that $\alpha^0 = 1, \alpha^1, \alpha^2, \dots, \alpha^{2^m-2}$ are **distinct elements** in $GF(2^m)$

Primitive Polynomials

- Let $\alpha \in GF(2^m)$ such that $\alpha^0 = 1, \alpha^1, \alpha^2, \dots, \alpha^{2^m-2}$ are **distinct elements** in $GF(2^m)$
- Then $GF(2^m) = \{0, 1, \alpha, \alpha^2, \dots, \alpha^{2^m-2}\}$ and α generates $GF(2^m)$

Primitive Polynomials

- Let $\alpha \in GF(2^m)$ such that $\alpha^0 = 1, \alpha^1, \alpha^2, \dots, \alpha^{2^m-2}$ are **distinct elements** in $GF(2^m)$
- Then $GF(2^m) = \{0, 1, \alpha, \alpha^2, \dots, \alpha^{2^m-2}\}$ and α generates $GF(2^m)$
- An irreducible polynomial $p(x)$ of degree m is said to be **primitive** if the **smallest positive integer n** for which $p(x)$ divides $x^n + 1$ is $n = 2^m - 1$, i.e., $p(x) | x^{2^m-1} + 1$

Primitive Polynomials

- Let $\alpha \in GF(2^m)$ such that $\alpha^0 = 1, \alpha^1, \alpha^2, \dots, \alpha^{2^m-2}$ are **distinct elements** in $GF(2^m)$
- Then $GF(2^m) = \{0, 1, \alpha, \alpha^2, \dots, \alpha^{2^m-2}\}$ and α generates $GF(2^m)$
- An irreducible polynomial $p(x)$ of degree m is said to be **primitive** if the **smallest positive integer n** for which $p(x)$ divides $x^n + 1$ is $n = 2^m - 1$, i.e., $p(x) | x^{2^m-1} + 1$
- Since $x^4 + x + 1 | x^{15} + 1$, $x^4 + x + 1$ is primitive. $x^4 + x^3 + x^2 + x + 1$ is not since $x^4 + x^3 + x^2 + x + 1 | x^5 + 1$

Primitive Polynomials

- Let $\alpha \in GF(2^m)$ such that $\alpha^0 = 1, \alpha^1, \alpha^2, \dots, \alpha^{2^m-2}$ are **distinct elements** in $GF(2^m)$
- Then $GF(2^m) = \{0, 1, \alpha, \alpha^2, \dots, \alpha^{2^m-2}\}$ and α generates $GF(2^m)$
- An irreducible polynomial $p(x)$ of degree m is said to be **primitive** if the **smallest positive integer n** for which $p(x)$ divides $x^n + 1$ is $n = 2^m - 1$, i.e., $p(x) \mid x^{2^m-1} + 1$
- Since $x^4 + x + 1 \mid x^{15} + 1$, $x^4 + x + 1$ is primitive. $x^4 + x^3 + x^2 + x + 1$ is not since $x^4 + x^3 + x^2 + x + 1 \mid x^5 + 1$
- Any root of $p(x)$ can be chosen as α to generate $GF(2^m)$

Three representations for the elements of $GF(2^4)$
generated by $p(x) = 1 + x + x^4$

Power representation		Polynomial representation				4-Tuple representation					
0	0					(0	0	0	0)		
1	1					(1	0	0	0)		
α		x				(0	1	0	0)		
α^2				x^2		(0	0	1	0)		
α^3						x^3		(0	0	0	1)
α^4	1	+	x			(1	1	0	0)		
α^5		x		+	x^2	(0	1	1	0)		
α^6				x^2		+	x^3	(0	0	1	1)
α^7	1	+	x			+	x^3	(1	1	0	1)
α^8	1			+	x^2			(1	0	1	0)
α^9		x				+	x^3	(0	1	0	1)
α^{10}	1	+	x	+	x^2			(1	1	1	0)
α^{11}		x		+	x^2	+	x^3	(0	1	1	1)
α^{12}	1	+	x	+	x^2	+	x^3	(1	1	1	1)
α^{13}	1			+	x^2	+	x^3	(1	0	1	1)
α^{14}	1					+	x^3	(1	0	0	1)

Computing $\alpha^8 \cdot \alpha^{10}$ and $\alpha^8 + \alpha^{10}$ in $GF(2^4)$

- $\alpha^8 \cdot \alpha^{10} = \alpha^{(8+10) \bmod 15} = \alpha^3$

Computing $\alpha^8 \cdot \alpha^{10}$ and $\alpha^8 + \alpha^{10}$ in $GF(2^4)$

- $\alpha^8 \cdot \alpha^{10} = \alpha^{(8+10) \bmod 15} = \alpha^3$

-

$$\begin{aligned}\alpha^8 + \alpha^{10} &= (1 + x^2) + (1 + x + x^2) \\ &= x = \alpha\end{aligned}$$

List of Primitive Polynomials

m		m	
3	$1 + x + x^3$	14	$1 + x + x^6 + x^{10} + x^{14}$
4	$1 + x + x^4$	15	$1 + x + x^{15}$
5	$1 + x^2 + x^5$	16	$1 + x + x^3 + x^{12} + x^{16}$
6	$1 + x + x^6$	17	$1 + x^3 + x^{17}$
7	$1 + x^3 + x^7$	18	$1 + x^7 + x^{18}$
8	$1 + x^2 + x^3 + x^4 + x^8$	19	$1 + x + x^2 + x^5 + x^{19}$
9	$1 + x^4 + x^9$	20	$1 + x^3 + x^{20}$
10	$1 + x^3 + x^{10}$	21	$1 + x^2 + x^{21}$
11	$1 + x^2 + x^{11}$	22	$1 + x + x^{22}$
12	$1 + x + x^4 + x^6 + x^{12}$	23	$1 + x^5 + x^{23}$
13	$1 + x + x^3 + x^4 + x^{13}$	24	$1 + x + x^2 + x^7 + x^{24}$

Singleton Bound on d_{min}

Singleton bound: $d_{min} \leq n - k + 1$ for an (n, k) linear block code

◀ Click for proof

Reed-Solomon Codes

- Reed-Solomon (RS) codes are a class of error-correcting codes invented by Reed and Solomon in 1960

Reed-Solomon Codes

- Reed-Solomon (RS) codes are a class of error-correcting codes invented by Reed and Solomon in 1960
- An (n, k) RS codes can correct up to $t = (n - k)/2$ errors ($n - k$ is even)

Reed-Solomon Codes

- Reed-Solomon (RS) codes are a class of error-correcting codes invented by Reed and Solomon in 1960
- An (n, k) RS codes can correct up to $t = (n - k)/2$ errors ($n - k$ is even)
- All vectors such as codewords and received vectors can be represented as **polynomials** where components of vectors are coefficients of the corresponding polynomials

Encoding of RS Codes (1)

- The construction of Reed-Solomon (RS) codes is to **evaluate the information polynomials** at all nonzero elements of finite field $GF(2^m)$

Encoding of RS Codes (1)

- The construction of Reed-Solomon (RS) codes is to **evaluate the information polynomials** at all nonzero elements of finite field $GF(2^m)$
- Let α be a primitive element in $GF(2^m)$ and let $n = 2^m - 1$

Encoding of RS Codes (1)

- The construction of Reed-Solomon (RS) codes is to **evaluate the information polynomials** at all nonzero elements of finite field $GF(2^m)$
- Let α be a primitive element in $GF(2^m)$ and let $n = 2^m - 1$
- Let $u(x) = u_0 + u_1x + \cdots + u_{k-1}x^{k-1}$ **be the information polynomial**, where $u_i \in GF(2^m)$ for all $0 \leq i \leq k-1$

Encoding of RS Codes (1)

- The construction of Reed-Solomon (RS) codes is to **evaluate the information polynomials** at all nonzero elements of finite field $GF(2^m)$
- Let α be a primitive element in $GF(2^m)$ and let $n = 2^m - 1$
- Let $u(x) = u_0 + u_1x + \cdots + u_{k-1}x^{k-1}$ **be the information polynomial**, where $u_i \in GF(2^m)$ for all $0 \leq i \leq k-1$
- The encoding is defined by the mapping $\rho : u(x) \rightarrow \mathbf{c}$ by

$$(c_0, c_1, \dots, c_{n-1}) = (u(1), u(\alpha), u(\alpha^2), \dots, u(\alpha^{n-1}))$$

Encoding of RS Codes (1)

- The construction of Reed-Solomon (RS) codes is to **evaluate the information polynomials** at all nonzero elements of finite field $GF(2^m)$
- Let α be a primitive element in $GF(2^m)$ and let $n = 2^m - 1$
- Let $u(x) = u_0 + u_1x + \cdots + u_{k-1}x^{k-1}$ **be the information polynomial**, where $u_i \in GF(2^m)$ for all $0 \leq i \leq k-1$
- The encoding is defined by the mapping $\rho : u(x) \rightarrow \mathbf{c}$ by

$$(c_0, c_1, \dots, c_{n-1}) = (u(1), u(\alpha), u(\alpha^2), \dots, u(\alpha^{n-1}))$$

- The minimum distance of an (n, k) RS code is $d_{min} = n - k + 1$

◀ Click for proof

Encoding of RS Codes (2)

$$\begin{aligned}
 u(\alpha^i) &= u_0 + u_1 \alpha^i + u_2 (\alpha^i)^2 + \cdots + u_{k-1} (\alpha^i)^{k-1} \\
 &= [u_0 \ u_1 \ \cdots \ u_{k-1}] \begin{bmatrix} 1 \\ \alpha^i \\ (\alpha^2)^i \\ \vdots \\ (\alpha^{k-1})^i \end{bmatrix}
 \end{aligned}$$

Encoding of RS Codes (2)

$$\begin{aligned}
 u(\alpha^i) &= u_0 + u_1 \alpha^i + u_2 (\alpha^i)^2 + \cdots + u_{k-1} (\alpha^i)^{k-1} \\
 &= [u_0 \ u_1 \ \cdots \ u_{k-1}] \begin{bmatrix} 1 \\ \alpha^i \\ (\alpha^2)^i \\ \vdots \\ (\alpha^{k-1})^i \end{bmatrix}
 \end{aligned}$$

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & \alpha & \cdots & \alpha^{n-1} \\ 1 & \alpha^2 & \cdots & (\alpha^{n-1})^2 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^{k-1} & \cdots & (\alpha^{n-1})^{k-1} \end{bmatrix}$$

Example of Encoding

- $GF(2^3)$ generated by $p(x) = 1 + x + x^3$

Power representation	Polynomial representation	Power representation	Polynomial representation			
0	0	α^3	1	+	x	
1	1	α^4			x	+
α		α^5	1	+	x	+
α^2		α^6	1			+
	x^2					x^2

Example of Encoding

- $GF(2^3)$ generated by $p(x) = 1 + x + x^3$

Power representation	Polynomial representation	Power representation	Polynomial representation		
0	0	α^3	1	+	x
1	1	α^4			x + x^2
α		α^5	1	+	x + x^2
α^2	x x^2	α^6	1		+ x^2

- Consider (7, 3) RS code over $GF(2^3)$

Example of Encoding

- $GF(2^3)$ generated by $p(x) = 1 + x + x^3$

Power representation	Polynomial representation	Power representation	Polynomial representation		
0	0	α^3	1	+	x
1	1	α^4			x + x^2
α	x	α^5	1	+	x + x^2
α^2	x^2	α^6	1		+ x^2

- Consider (7, 3) RS code over $GF(2^3)$
- Let the data sequence be 000 010 110. The data map to symbols $0 \ \alpha \ \alpha^3$. Then $u(x) = \alpha x + \alpha^3 x^2$

Example of Encoding

- $GF(2^3)$ generated by $p(x) = 1 + x + x^3$

Power representation	Polynomial representation	Power representation	Polynomial representation
0	0	α^3	$1 + x$
1	1	α^4	$x + x^2$
α	x	α^5	$1 + x + x^2$
α^2	x^2	α^6	$1 + x^2$

- Consider (7, 3) RS code over $GF(2^3)$
- Let the data sequence be 000 010 110. The data map to symbols $0 \ \alpha \ \alpha^3$. Then $u(x) = \alpha x + \alpha^3 x^2$
-

$$\begin{aligned}
 (c_0, c_1, \dots, c_6) &= (u(1), u(\alpha), u(\alpha^2), \dots, u(\alpha^6)) \\
 &= (\alpha + \alpha^3, \alpha^2 + \alpha^5, \alpha^3 + \alpha^7, \alpha^4 + \alpha^9, \alpha^5 + \alpha^{11}, \alpha^6 + \alpha^{13}, \alpha^7 + \alpha^{15}) \\
 &= (1, \alpha^3, \alpha, \alpha, 1, 0, \alpha^3)
 \end{aligned}$$

Example of Encoding

- $GF(2^3)$ generated by $p(x) = 1 + x + x^3$

Power representation	Polynomial representation	Power representation	Polynomial representation
0	0	α^3	1 + x
1	1	α^4	x + x^2
α	x	α^5	1 + x + x^2
α^2	x^2	α^6	1 + x^2

- Consider (7, 3) RS code over $GF(2^3)$
- Let the data sequence be 000 010 110. The data map to symbols $0 \ \alpha \ \alpha^3$. Then $u(x) = \alpha x + \alpha^3 x^2$
-

$$\begin{aligned}
 (c_0, c_1, \dots, c_6) &= (u(1), u(\alpha), u(\alpha^2), \dots, u(\alpha^6)) \\
 &= (\alpha + \alpha^3, \alpha^2 + \alpha^5, \alpha^3 + \alpha^7, \alpha^4 + \alpha^9, \alpha^5 + \alpha^{11}, \alpha^6 + \alpha^{13}, \alpha^7 + \alpha^{15}) \\
 &= (1, \alpha^3, \alpha, \alpha, 1, 0, \alpha^3)
 \end{aligned}$$

- The binary sequence for the codeword is

100 110 010 010 100 000 110

Decoding Procedure

- The RS codes decoding has four steps:

Decoding Procedure

- The RS codes decoding has four steps:
 - 1 Syndrome computation

Decoding Procedure

- The RS codes decoding has four steps:
 - 1 Syndrome computation
 - 2 Solving the key equation for the error-locator polynomial $\Lambda(x)$

Decoding Procedure

- The RS codes decoding has four steps:
 - 1 Syndrome computation
 - 2 Solving the key equation for the error-locator polynomial $\Lambda(x)$
 - 3 Searching error locations given the $\Lambda(x)$ polynomial by simply finding the inverse roots (Chien search)

Decoding Procedure

- The RS codes decoding has four steps:
 - 1 Syndrome computation
 - 2 Solving the key equation for the error-locator polynomial $\Lambda(x)$
 - 3 Searching error locations given the $\Lambda(x)$ polynomial by simply finding the inverse roots (Chien search)
 - 4 Determine the error magnitude at each error location by error-evaluator polynomial $\Omega(x)$ (Forney's formula)

Decoding Procedure

- The RS codes decoding has four steps:
 - ① Syndrome computation
 - ② Solving the key equation for the error-locator polynomial $\Lambda(x)$
 - ③ Searching error locations given the $\Lambda(x)$ polynomial by simply finding the inverse roots (Chien search)
 - ④ Determine the error magnitude at each error location by error-evaluator polynomial $\Omega(x)$ (Forney's formula)
- In this talk, we only focus on the first two steps

Roots of Code Polynomials of Reed-Solomon Codes

It can be shown that $\alpha^1, \alpha^2, \dots, \alpha^{2t}$ are roots of code polynomials

$$c(x) = c_0 + c_1x + c_2x^2 + \cdots + c_{n-1}x^{n-1},$$

where $t = \frac{n-k}{2}$

► Click for proof

Syndrome Computation (1)

- Let $y(x)$ be the received vector. Then define the syndrome S_j , $1 \leq j \leq 2t$, as follows:

$$\begin{aligned}
 S_j &= y(\alpha^j) = c(\alpha^j) + e(\alpha^j) = e(\alpha^j) \\
 &= \sum_{i=0}^{n-1} e_i (\alpha^j)^i = \sum_{k=1}^v e_{i_k} \alpha^{i_k j}
 \end{aligned}$$

Syndrome Computation (1)

- Let $y(x)$ be the received vector. Then define the syndrome S_j , $1 \leq j \leq 2t$, as follows:

$$\begin{aligned} S_j &= y(\alpha^j) = c(\alpha^j) + e(\alpha^j) = e(\alpha^j) \\ &= \sum_{i=0}^{n-1} e_i (\alpha^j)^i = \sum_{k=1}^v e_{i_k} \alpha^{i_k j} \end{aligned}$$

- n is the code length and it is assumed that v errors occurred in locations corresponding to time indexes $i_1, i_2, \dots, i_v$.

Syndrome Computation (1)

- Let $y(x)$ be the received vector. Then define the syndrome S_j , $1 \leq j \leq 2t$, as follows:

$$\begin{aligned} S_j &= y(\alpha^j) = c(\alpha^j) + e(\alpha^j) = e(\alpha^j) \\ &= \sum_{i=0}^{n-1} e_i (\alpha^j)^i = \sum_{k=1}^v e_{i_k} \alpha^{i_k j} \end{aligned}$$

- n is the code length and it is assumed that v errors occurred in locations corresponding to time indexes $i_1, i_2, \dots, i_v$.
- For ease of notation we reformulate the syndromes as

$$S_j = \sum_{k=1}^v Y_k X_k^j, \text{ for } 1 \leq j \leq 2t,$$

where $Y_k = e_{i_k}$ and $X_k = \alpha^{i_k}$

Syndrome Computation (2)

- The system of equations for syndromes is

$$S_1 = Y_1X_1 + Y_2X_2 + \cdots + Y_vX_v$$

$$S_2 = Y_1X_1^2 + Y_2X_2^2 + \cdots + Y_vX_v^2$$

$$S_3 = Y_1X_1^3 + Y_2X_2^3 + \cdots + Y_vX_v^3$$

$$\vdots$$

$$S_{2t} = Y_1X_1^{2t} + Y_2X_2^{2t} + \cdots + Y_vX_v^{2t}$$

Key Equation (1)

- Define the infinite degree syndrome polynomial (though we **only know the first $2t$ coefficients**) as

$$\begin{aligned}
 S(x) &= \sum_{j=0}^{\infty} S_{j+1} x^j = \sum_{j=0}^{\infty} x^j \left(\sum_{k=1}^v Y_k X_k^{j+1} \right) \\
 &= \sum_{k=1}^v \frac{Y_k X_k}{1 - x X_k}
 \end{aligned}$$

Key Equation (1)

- Define the infinite degree syndrome polynomial (though we **only know the first $2t$ coefficients**) as

$$\begin{aligned} S(x) &= \sum_{j=0}^{\infty} S_{j+1} x^j = \sum_{j=0}^{\infty} x^j \left(\sum_{k=1}^v Y_k X_k^{j+1} \right) \\ &= \sum_{k=1}^v \frac{Y_k X_k}{1 - x X_k} \end{aligned}$$

- The error-locator polynomial is

$$\Lambda(x) = (1 - xX_1)(1 - xX_2) \cdots (1 - xX_v) = \Lambda_0 + \sum_{i=1}^v \Lambda_i x^i,$$

where $\Lambda_0 = 1$

Key Equation (2)

- Define the error-evaluator polynomial as

$$\Omega(x) \triangleq \Lambda(x)S(x) = \sum_{k=1}^v Y_k X_k \prod_{\substack{j=1 \\ j \neq k}}^v (1 - xX_j)$$

- $\deg(\Omega(x)) < \deg(\Lambda(x)) = v$

Key Equation (2)

- Define the error-evaluator polynomial as

$$\Omega(x) \triangleq \Lambda(x)S(x) = \sum_{k=1}^v Y_k X_k \prod_{\substack{j=1 \\ j \neq k}}^v (1 - xX_j)$$

- $\deg(\Omega(x)) < \deg(\Lambda(x)) = v$
- Actually we only know the first $2t$ terms of $S(x)$ such that we have the key equation:

$$\Lambda(x)S(x) \equiv \Omega(x) \pmod{x^{2t}} \quad (2)$$

Key Equation (2)

- Define the error-evaluator polynomial as

$$\Omega(x) \triangleq \Lambda(x)S(x) = \sum_{k=1}^v Y_k X_k \prod_{\substack{j=1 \\ j \neq k}}^v (1 - xX_j)$$

- $\deg(\Omega(x)) < \deg(\Lambda(x)) = v$
- Actually we only know the first $2t$ terms of $S(x)$ such that we have the key equation:

$$\Lambda(x)S(x) \equiv \Omega(x) \pmod{x^{2t}} \quad (2)$$

- Extended Euclidean algorithm to find $(g_i(x), r_i(x))$ in $g_i(x)b(x) = r_i(x) \pmod{a(x)}$

Key Equation (2)

- Define the error-evaluator polynomial as

$$\Omega(x) \triangleq \Lambda(x)S(x) = \sum_{k=1}^v Y_k X_k \prod_{\substack{j=1 \\ j \neq k}}^v (1 - xX_j)$$

- $\deg(\Omega(x)) < \deg(\Lambda(x)) = v$
- Actually we only know the first $2t$ terms of $S(x)$ such that we have the key equation:

$$\Lambda(x)S(x) \equiv \Omega(x) \pmod{x^{2t}} \quad (2)$$

- Extended Euclidean algorithm to find $(g_i(x), r_i(x))$ in $g_i(x)b(x) = r_i(x) \pmod{a(x)}$
- $S(x) \rightarrow b(x), x^{2t} \rightarrow a(x)$ Find $(g_i(x), r_i(x)) \rightarrow (\Lambda(x), \Omega(x))$

Extended Euclidean Algorithm for Polynomials (1)

- Considering a simple example, where $a(x) = x^5 + 1$ and $b(x) = x^3 + 1$.
Then

$$x^5 + 1 = x^2(x^3 + 1) + (x^2 + 1)$$

$$x^3 + 1 = x(x^2 + 1) + (x + 1)$$

$$x^2 + 1 = (x + 1)(x + 1) + 0$$

$$x + 1 = d(x)$$

Extended Euclidean Algorithm for Polynomials (1)

- Considering a simple example, where $a(x) = x^5 + 1$ and $b(x) = x^3 + 1$.
Then

$$x^5 + 1 = x^2(x^3 + 1) + (x^2 + 1)$$

$$x^3 + 1 = x(x^2 + 1) + (x + 1)$$

$$x^2 + 1 = (x + 1)(x + 1) + 0$$

$$x + 1 = d(x)$$

- The useful aspect of this process is that, at each iteration, a set of polynomials $f_i(x)$, $g_i(x)$, and $r_i(x)$ are generated such that

$$f_i(x)a(x) + g_i(x)b(x) = r_i(x) \quad (3)$$

Extended Euclidean Algorithm for Polynomials (2)

- A way to obtain $f_i(x)$ and $g_i(x)$ is as follows

Extended Euclidean Algorithm for Polynomials (2)

- A way to obtain $f_i(x)$ and $g_i(x)$ is as follows
- Define $q_i(x)$ to be the quotient polynomial that is produced by dividing $r_{i-2}(x)$ by $r_{i-1}(x)$. Then, for $i \geq 1$,

$$r_i(x) = r_{i-2}(x) - q_i(x)r_{i-1}(x)$$

$$f_i(x) = f_{i-2}(x) - q_i(x)f_{i-1}(x)$$

$$g_i(x) = g_{i-2}(x) - q_i(x)g_{i-1}(x),$$

where the initial values are

$$f_{-1}(x) = g_0(x) = 1$$

$$f_0(x) = g_{-1}(x) = 0$$

$$r_{-1}(x) = a(x)$$

$$r_0(x) = b(x)$$

(4)

Sugiyama Algorithm (1)

- The Sugiyama algorithm utilizes extended Euclidean algorithm to solve the key equation. Hence, the Sugiyama algorithm is also called Euclidean algorithm

Sugiyama Algorithm (1)

- The Sugiyama algorithm utilizes extended Euclidean algorithm to solve the key equation. Hence, the Sugiyama algorithm is also called Euclidean algorithm
- $f_i(x)a(x) + g_i(x)b(x) = r_i(x)$ in (3) can be written as

$$g_i(x)b(x) \equiv r_i(x) \pmod{a(x)}$$

Sugiyama Algorithm (1)

- The Sugiyama algorithm utilizes extended Euclidean algorithm to solve the key equation. Hence, the Sugiyama algorithm is also called Euclidean algorithm
- $f_i(x)a(x) + g_i(x)b(x) = r_i(x)$ in (3) can be written as

$$g_i(x)b(x) \equiv r_i(x) \pmod{a(x)}$$

- Comparing $\Lambda(x)S(x) \equiv \Omega(x) \pmod{x^{2t}}$ in (2) with the above equation, they are equivalent when

$$\begin{aligned} a(x) &= x^{2t}, \quad b(x) = S(x) \\ g_i(x) &= \Lambda_i(x), \quad r_i(x) = \Omega_i(x) \end{aligned}$$

Sugiyama Algorithm (1)

- The Sugiyama algorithm utilizes extended Euclidean algorithm to solve the key equation. Hence, the Sugiyama algorithm is also called Euclidean algorithm
- $f_i(x)a(x) + g_i(x)b(x) = r_i(x)$ in (3) can be written as

$$g_i(x)b(x) \equiv r_i(x) \pmod{a(x)}$$

- Comparing $\Lambda(x)S(x) \equiv \Omega(x) \pmod{x^{2t}}$ in (2) with the above equation, they are equivalent when

$$\begin{aligned} a(x) &= x^{2t}, \quad b(x) = S(x) \\ g_i(x) &= \Lambda_i(x), \quad r_i(x) = \Omega_i(x) \end{aligned}$$

- The Euclidean algorithm produces a sequence of solutions to the key equation

Sugiyama Algorithm (2)

Summary of Sugiyama Decoding algorithm

- ① Apply Euclidean algorithm to $a(x) = x^{2t}$ and $b(x) = S(x)$
- ② Use the initial conditions of (4) ▶ Go to (4)
- ③ Stop when $\deg[r_\ell(x)] < t$ ▶ Details of stopping rule
- ④ Set $\Lambda(x) = g_\ell(x)$ and $\Omega(x) = r_\ell(x)$

Example of Decoding-Syndrome Computation

- $GF(2^3)$ generated by $p(x) = 1 + x + x^3$

Power representation	Polynomial representation	Power representation	Polynomial representation				
0	0	α^3	1	+	x		
1	1	α^4			x	+	x^2
α		α^5	1	+	x	+	x^2
α^2		α^6	1			+	x^2
	x^2						

Example of Decoding-Syndrome Computation

- $GF(2^3)$ generated by $p(x) = 1 + x + x^3$

Power representation	Polynomial representation	Power representation	Polynomial representation				
0	0	α^3	1	+	x		
1	1	α^4			x	+	x^2
α		α^5	1	+	x	+	x^2
α^2		α^6	1			+	x^2
	x						
	x^2						

- Let the received polynomial is

$$y(x) = 1 + \alpha^5 x + \alpha^2 x^2 + \alpha x^3 + x^4 + \alpha^3 x^6$$

Example of Decoding-Syndrome Computation

- $GF(2^3)$ generated by $p(x) = 1 + x + x^3$

Power representation	Polynomial representation	Power representation	Polynomial representation		
0	0	α^3	1	+	x
1	1	α^4			$x + x^2$
α	x	α^5	1	+	$x + x^2$
α^2	x^2	α^6	1		$+ x^2$

- Let the received polynomial is

$$y(x) = 1 + \alpha^5 x + \alpha^2 x^2 + \alpha x^3 + x^4 + \alpha^3 x^6$$

-

$$S_1 = y(\alpha) = \alpha^4, \quad S_2 = y(\alpha^2) = \alpha^2$$

$$S_3 = y(\alpha^3) = \alpha^2, \quad S_4 = y(\alpha^4) = \alpha$$

Example of Decoding-Sugiyama Algorithm

- We have $a(x) = x^4$ and $b(x) = S(x) = \alpha^4 + \alpha^2x + \alpha^2x^2 + \alpha x^3$
- Applying Extended Euclidean algorithm:

i	$\Lambda_i(x)(g_i(x))$	$\Omega_i(x)(r_i(x))$	$q_i(x)$
-1	0	x^4	—
0	1	$S(x)$	—
1	$\alpha^6x + 1$	$\alpha^4x^2 + \alpha^5x + \alpha^4$	$\alpha^6x + 1$
2	$\alpha^3x^2 + \alpha^4x + 1$	$\alpha^4x + \alpha^4$	α^4x

- Thus, $\Lambda(x) = \alpha^3x^2 + \alpha^4x + 1$. By performing [Chien search](#) we can find the roots of $\Lambda(x)$ are $\alpha^6 = \alpha^{-1}$ and $\alpha^5 = \alpha^{-2}$. The errors are in x and x^2
- Performing [Forney's Formula](#), we can determine the error magnitude for each error location as α^2 for x and α^4 for x^2

Properties of RS Codes

- $(c_0, c_1, \dots, c_{n-1}) = (u(1), u(\alpha), u(\alpha^2), \dots, u(\alpha^{n-1}))$

- $$\mathbf{G} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & \alpha & \dots & \alpha^{n-1} \\ 1 & \alpha^2 & \dots & (\alpha^{n-1})^2 \\ & & \vdots & \\ 1 & \alpha^{k-1} & \dots & (\alpha^{n-1})^{k-1} \end{bmatrix}$$

Properties of RS Codes

- $(c_0, c_1, \dots, c_{n-1}) = (u(1), u(\alpha), u(\alpha^2), \dots, u(\alpha^{n-1}))$

- $$\mathbf{G} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & \alpha & \dots & \alpha^{n-1} \\ 1 & \alpha^2 & \dots & (\alpha^{n-1})^2 \\ & & \vdots & \\ 1 & \alpha^{k-1} & \dots & (\alpha^{n-1})^{k-1} \end{bmatrix}$$

- Any k columns of $\mathbf{G}$ are linearly independent

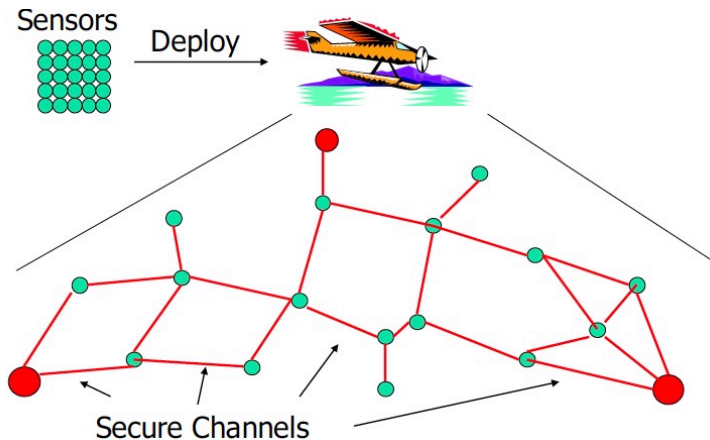
Properties of RS Codes

- $$(c_0, c_1, \dots, c_{n-1}) = (u(1), u(\alpha), u(\alpha^2), \dots, u(\alpha^{n-1}))$$

- $$\mathbf{G} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & \alpha & \dots & \alpha^{n-1} \\ 1 & \alpha^2 & \dots & (\alpha^{n-1})^2 \\ & & \vdots & \\ 1 & \alpha^{k-1} & \dots & (\alpha^{n-1})^{k-1} \end{bmatrix}$$

- Any k columns of $\mathbf{G}$ are linearly independent
- Any k codeword symbols can generate the rest $n - k$ codeword symbols

Secure Wireless Sensor Networks



Key Distribution System (KDS)

- During an initialization stage, a trusted server generates and distributes secret data values to users

Key Distribution System (KDS)

- During an initialization stage, a trusted server generates and distributes secret data values to users
- Any pair of users may compute a shared key unknown to all others (aside from the server)

j-Secure KDS

- Given a specified pair of users

j -Secure KDS

- Given a specified pair of users
- Any coalition of j or fewer users can do no better at computing the key shared by the two than a party which guesses the key without any secret data values

Blom's Scheme

- Public matrix $\mathbf{G}$ (The given generator matrix for RS codes)

Blom's Scheme

- Public matrix $\mathbf{G}$ (The given generator matrix for RS codes)
- Private symmetric matrix $\mathbf{D}$

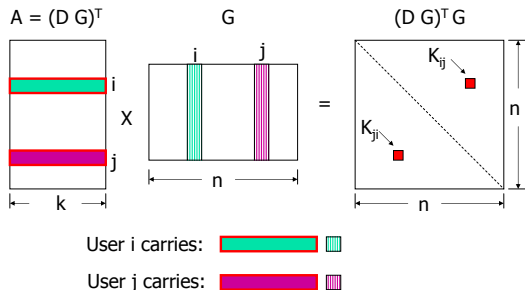
Blom's Scheme

- Public matrix $\mathbf{G}$ (The given generator matrix for RS codes)
- Private symmetric matrix $\mathbf{D}$
- Let $\mathbf{A} = (\mathbf{D}\mathbf{G})^T$. Then $\mathbf{A}\mathbf{G}$ is symmetric.

$$\mathbf{A}\mathbf{G} = (\mathbf{D}\mathbf{G})^T\mathbf{G} = \left(\mathbf{G}^T\mathbf{D}\mathbf{G}\right)^T = ((\mathbf{D}\mathbf{G})^T\mathbf{G})^T = (\mathbf{A}\mathbf{G})^T$$

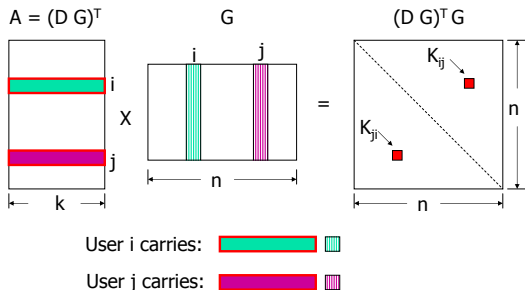
Blom's Scheme (cont'd)

- Blom's scheme is a $(k-1)$ -secure KDS



Blom's Scheme (cont'd)

- Blom's scheme is a $(k-1)$ -secure KDS
- Each user carries k symbols (keys)



Reference

- R. Blom, “An Optimal Class of Symmetric Key Generation Systems,” *Advances in Cryptology-Proceedings of EUROCRYPT 84 (LNCS 209)*, pp. 335-338, 1985
- W. Du, J. Deng, **Y. S. Han**, and P. K. Varshney, “A Pairwise Key Pre-distribution Scheme for Wireless Sensor Networks,” *Proceedings of 10th ACM Conference on Computer and Communications Security (CCS2003)*, Washington DC, October, 2003, pp. 42-51. **(Cited for more than 2400 times, 2013 ACM CCS Test-of-Time Award)**

Conclusion

- Reed-Solomon (RS) codes are a class of error-correcting codes that have been applied on many important applications

Conclusion

- Reed-Solomon (RS) codes are a class of error-correcting codes that have been applied on many important applications
- The basic concept for finite fields and RS codes can be introduced without advanced algebra background

Conclusion

- Reed-Solomon (RS) codes are a class of error-correcting codes that have been applied on many important applications
- The basic concept for finite fields and RS codes can be introduced without advanced algebra background
- Extended Euclidean algorithm is the key technology for decoding of RS codes

The Definition of Fields

- Let F be a set of elements on which two binary operations, called addition “+” and multiplication “ $\cdot$ ”, are defined. The set F together with the two binary operations $+$ and $\cdot$ is a field if the following conditions are satisfied:
 - (Closure)** For all $a, b \in F$, $a + b$ and $a \cdot b$ are in F
 - (Associativity)** For all $a, b, c \in F$, $a + (b + c) = (a + b) + c$ and $a \cdot (b \cdot c) = (a \cdot b) \cdot c$
 - (Commutativity)** For all $a, b \in F$, $a + b = b + a$ and $a \cdot b = b \cdot a$
 - (Identity)** There exists an element of F , denoted by 0 (additive identity), such that for all $a \in F$, $a + 0 = a$; there exists an element of F , denoted by 1 (multiplicative identity), such that for all $a \in F$, $a \cdot 1 = a$
 - (Inverse)** For all $a \in F$, there exists an element of F , denoted by $-a$ (additive inverse), such that $a + (-a) = 0$; for all nonzero $a \in F$, there exists an element of F , denoted by a^{-1} (multiplicative inverse), such that $a \cdot a^{-1} = 1$
 - (Distributivity)** For all $a, b, c \in F$, $a \cdot (b + c) = a \cdot b + a \cdot c$

Return

Proof of Property of Finite Fields

- Let a be any nonzero elements in $GF(2^m)$ and $b_i, 1 \leq i \leq 2^m - 1$, are all nonzero elements. Then $a \cdot b_i, 1 \leq i \leq 2^m - 1$, are all nonzero elements in different order (if $b_i \neq b_j$, then $a \cdot b_i \neq a \cdot b_j$)
- We have

$$\begin{aligned}(a \cdot b_1) \cdot (a \cdot b_2) \cdots (a \cdot b_{2^m-1}) &= b_1 \cdot b_2 \cdots b_{2^m-1} \\ \Rightarrow a^{2^m-1} \cdot (b_1 \cdot b_2 \cdots b_{2^m-1}) &= b_1 \cdot b_2 \cdots b_{2^m-1}\end{aligned}$$

Hence, $a^{2^m-1} = 1$

[Return to property of finite fields](#)

Proof of Minimum Distances of RS Codes

Since $u(x)$ has at most $k - 1$ roots, there are at most $k - 1$ zero positions in each nonzero codeword. Hence, $d_{min} \geq n - k + 1$. By the Singleton bound, $d_{min} \leq n - k + 1$. So $d_{min} = n - k + 1$

[Return to encoding of RS codes](#)

Proof of Roots of Code Polynomials of RS Codes

For any $\alpha^\ell \in GF(2^m)$

$$\begin{aligned} c(\alpha^\ell) &= c_0 + c_1\alpha^\ell + c_2(\alpha^\ell)^2 + \cdots + c_{n-1}(\alpha^\ell)^{n-1} \\ &= \sum_{i=0}^{n-1} c_i(\alpha^\ell)^i = \sum_{i=0}^{n-1} \left(\sum_{j=0}^{k-1} u_j(\alpha^i)^j \right) (\alpha^\ell)^i = \sum_{j=0}^{k-1} u_j \sum_{i=0}^{n-1} (\alpha^{\ell+j})^i \end{aligned}$$

If $\alpha^{\ell+j} \neq 1$, i.e. $\ell + j \neq n = 2^m - 1$, then $\sum_{i=0}^{n-1} (\alpha^{\ell+j})^i = 0$. Since $j = 0, 1, \dots, k-1$, $c(\alpha^\ell) = 0$ for $\ell = 1, 2, \dots, n - k = 2t$

► Return to Roots of Code Polynomials of RS Codes

Stopping Rule in Sugiyama Algorithm

- Two useful properties of the Extended Euclidean algorithm:

$$\deg[r_i(x)] < \deg[r_{i-1}(x)] ; \deg[g_i(x)] + \deg[r_{i-1}(x)] = \deg[a(x)]$$

- By the property we have

$$\deg[g_i(x)] + \deg[r_{i-1}(x)] = 2t$$

and

$$\deg[g_i(x)] + \deg[r_i(x)] < 2t$$

- If $v \leq t$, then

$$\deg[\Omega(x)] < \deg[\Lambda(x)] \leq t \quad (\deg[r_\ell(x)] < \deg[g_\ell(x)] \leq t)$$

- There exists only one polynomial $\Lambda(x)$ with degree no greater than t which satisfies the key equation
- If $\deg[r_{\ell-1}(x)] \geq t$, then $\deg[g_\ell(x)] \leq t$. Since $\deg[r_\ell(x)] < t$, $\deg[g_{\ell+1}(x)] > t$
- The results at the ℓ th step provide the only solution to the key equation that is of interest

[Return to Sugiyama algorithm](#)

Chien Search (1)

- The next important decoding step is to find the actual error locations $X_1 = \alpha^{i_1}, X_2 = \alpha^{i_2}, \dots, X_v = \alpha^{i_v}$
- Note that $\Lambda(x)$ has roots $X_1^{-1} = \alpha^{-i_1}, X_2^{-1} = \alpha^{-i_2}, \dots, X_v^{-1} = \alpha^{-i_v}$
- Observe that an error occurs in position i if and only if $\Lambda(\alpha^{-i}) = 0$ or

$$\sum_{k=0}^v \Lambda_k \alpha^{-ik} = 0$$

- Then

$$\Lambda(\alpha^{-(i-1)}) = \sum_{k=0}^v \Lambda_k \alpha^{-ik+k} = \sum_{k=0}^v \left(\Lambda_k \alpha^{-ik} \right) \alpha^k$$

- This suggests that the potential error locations are tested in succession starting with time index $n - 1$

Chien Search (2)

- 1 Summing all terms of $\Lambda(\alpha^{-i})$ at index i tests to see whether $\Lambda(\alpha^{-i}) = 0$
- 2 Then to test at index $i - 1$ only requires multiplying the k th term of $\Lambda(\alpha^{-i})$ by α^k for all k and summing all terms again
- 3 This procedure is repeated until index 0 is reached
- 4 The initial value for k th term is $\Lambda_k \alpha^{-nk} = \Lambda_k$

Example of Chien Search

- $\Lambda(x) = 1 + \alpha^4 x + \alpha^3 x^2$ and $n = 7$

-

$$\Lambda(\alpha^{-6}) = 1 + \alpha^4 \alpha + \alpha^3 (\alpha)^2 = 1 + \alpha^5 + \alpha^5 = 1$$

$$\Lambda(\alpha^{-5}) = 1 + \alpha^5 \alpha + \alpha^5 (\alpha)^2 = 1 + \alpha^6 + 1 = \alpha^6$$

$$\Lambda(\alpha^{-4}) = 1 + \alpha^6 \alpha + 1 \cdot (\alpha)^2 = 1 + 1 + \alpha^2 = \alpha^2$$

$$\Lambda(\alpha^{-3}) = 1 + 1 \cdot \alpha + \alpha^2 (\alpha)^2 = 1 + \alpha + \alpha^4 = \alpha^6$$

$$\Lambda(\alpha^{-2}) = 1 + \alpha \alpha + \alpha^4 (\alpha)^2 = 1 + \alpha^2 + \alpha^6 = 0$$

$$\Lambda(\alpha^{-1}) = 1 + \alpha^2 \alpha + \alpha^6 (\alpha)^2 = 1 + \alpha^3 + \alpha = 0$$

$$\Lambda(\alpha^0) = 1 + \alpha^3 \alpha + \alpha (\alpha)^2 = 1 + \alpha^4 + \alpha^3 = \alpha^2$$

◀ Return

Forney's Formula (1)

- For RS codes one still needs to determine the error magnitude for each error location
- These values, $Y_1, Y_2, \dots, Y_v$, can be obtained by utilizing the error-evaluator polynomial. This step is known as *Forney's formula*
- By substituting $X_k^{-1} = \alpha^{-i_k}$ into the error-evaluator polynomial we have

$$\Omega(X_k^{-1}) = Y_k X_k \prod_{\substack{j=1 \\ j \neq k}}^v (1 - X_k^{-1} X_j)$$

Forney's Formula (2)

- By taking the formal derivative of $\Lambda(x)$ and also evaluating it at $x = X_k^{-1}$ we have

$$\begin{aligned}\Lambda'(X_k^{-1}) &= -X_k \prod_{\substack{j=1 \\ j \neq k}}^v (1 - X_k^{-1} X_j) \\ &= \frac{-1}{Y_k} \Omega(X_k^{-1})\end{aligned}$$

- Thus the error magnitude Y_k is given by

$$Y_k = -\frac{\Omega(X_k^{-1})}{\Lambda'(X_k^{-1})} = -\frac{\Omega(\alpha^{-i_k})}{\Lambda'(\alpha^{-i_k})} \quad (5)$$

Example of Forney's Formula

- $\Lambda(x) = 1 + \alpha^4 x + \alpha^3 x^2$, $\Lambda'(x) = \alpha^4$, and $\Omega(x) = \alpha^4 + \alpha^4 x$
- By Chien search, the error locations are $k = 1, 2$
-

$$Y_1 = \frac{\Omega(\alpha^{-1})}{\Lambda'(\alpha^{-1})} = \frac{\alpha^4 + \alpha^3}{\alpha^4} = 1 + \alpha^6 = \boxed{\alpha^2}$$

$$Y_2 = \frac{\Omega(\alpha^{-2})}{\Lambda'(\alpha^{-2})} = \frac{\alpha^4 + \alpha^2}{\alpha^4} = 1 + \alpha^5 = \boxed{\alpha^4}$$

◀ Return